

DSL Verbal Interface & Natural Language Translation

Voice-First Agent Orchestration for Claude Code Version: 1.0.0 Date: 2025-10-19 Use Case: Hands-free operation, accessibility, working parent mode ---

Table of Contents

1. [Introduction](#) 2. [Natural Language to DSL Mapping](#) 3. [Voice-Friendly Syntax](#) 4. [Speech Recognition Patterns](#) 5. [Natural Language Compiler](#) 6. [Practical Examples](#) 7. [Voice Mode Integration](#) 8. [Accessibility Features](#) ---

1. Introduction

1.1 The Problem

Complex DSL syntax is powerful but impractical for: - **Voice input:** Hard to say `"/ctx7 ll /deep - > synthesize"` - **Hands-free mode:** Working with kids, multitasking - **Accessibility:** Screen readers, motor impairments - **Cognitive load:** Easier to think in natural language

1.2 The Solution

Three-tier translation:

```

Natural Language (what you SAY)
      ↓
Verbal DSL (speakable syntax)
      ↓
Formal DSL (mathematical notation)
      ↓
Execution

```

Example:

```

SAY:      "Run deep research and context lookup in parallel,
           then synthesize findings on DSL design"

VERBAL: "parallel: deep research, context lookup
        then: synthesize with task DSL design"

FORMAL: (/deep || /ctx7) -> synthesize : "DSL design"

EXECUTES: Parallel research → synthesis

```

2. Natural Language to DSL Mapping

2.1 Core Patterns

Natural Language	Verbal DSL	Formal DSL	Meaning
"then" "then"	->	Sequential	
"and" "and"		Parallel	with with +
"on" "for" "task"	:	Specification	"equals" "is" =
"if...else" "if...else"	? :	Conditional	"try...otherwise" "try...fallback" ↺ :
			Retry

2.2 Agent Names (Speakable)

Problem: /ctx7 is hard to say **Solution:** Speakable aliases | Formal | Verbal Alias | Natural Speech

Formal	Verbal Alias	Natural Speech
/ctx7	"context lookup" "context seven"	
/deep	"deep research" "deep researcher"	/meta-skill-builder "skill builder"

"meta skill builder" | | /api-architect | "API architect" | "A P I architect" | | frontend-specialist | "frontend" | "front end specialist" |

2.3 Complexity Levels (Speakable)

Level 1: Simple Commands **Natural:**

```
"API architect, design a REST API"
```

Verbal:

```
API architect task: design a REST API
```

Formal:

```
api-architect : "design a REST API"
```

--- #### Level 2: Binary Operations **Natural:**

```
"Run research, then design, then implement"
```

Verbal:

```
research, then design, then implement
```

Formal:

```
research -> design -> implement
```

--- **Natural:**

```
"Run security scan and performance test and code review together"
```

Verbal:

```
parallel: security scan, performance test, code review
```

Formal:

```
security_scan || performance_test || code_review
```

--- Natural:

```
"API architect with REST patterns and PostgreSQL skills"
```

Verbal:

```
API architect with REST patterns with PostgreSQL
```

Formal:

```
api-architect + rest-patterns + postgresql
```

--- ##### Level 3: Parallel Streams Natural:

```
"Run three research streams in parallel:  
first stream is deep research plus context lookup,  
second stream is orchestration tools,  
third stream is meta builders.  
All working on DSL design task."
```

Verbal:

```
parallel streams:  
  stream 1: deep research and context lookup  
  stream 2: orchestration tools  
  stream 3: meta builders  
task: DSL design
```

Formal:

```
(  
  /deep + /ctx7 ||  
  /orch + /wflw ||  
  /meta-skill-builder  
) : "DSL design"
```

--- ##### Level 4: Complex Orchestration **Natural:**

```
"Run tests. If all pass, deploy to production.  
Otherwise, fix failures and retry."
```

Verbal:

```
run tests  
if pass:  
  deploy production  
else:  
  fix failures, then retry
```

Formal:

```
test_suite ->  
if(all_pass)  
  deploy_production  
: (fix_failures -> retest)
```

--- **Natural:**

```
"Research, then run design and implementation in parallel,  
then integrate everything."
```

Verbal:

```
research  
then parallel: design, implementation  
then integrate
```

Formal:

```
research ->
(design || implementation) ->
integrate
```

--- ##### Level 5: Named Workflows **Natural:**

```
"Create a workflow called microservice dev.
Steps: research domain, then design API and database together,
then implement, then test."
```

Verbal:

```
workflow microservice dev:
  research domain
  then parallel: design API, design database
  then implement
  then test
```

Formal:

```
workflow microservice_dev:
  research_domain ->
  (design_api || design_database) ->
  implement ->
  test
```

--- ##### Level 6: Meta-Programming **Natural:**

```
"Generate a workflow for any entity type.
The workflow should: design schema, create handlers,
write tests, all customized for the entity."
```

Verbal:

```
meta workflow for entity type:
  generate schema for entity
  generate handlers for entity
  generate tests for entity
```

Formal:

```
meta_workflow(Entity) = λe. {  
  generate_schema(e) ->  
  generate_handlers(e) ->  
  generate_tests(e)  
}
```

3. Voice-Friendly Syntax

3.1 Speakable Keywords

Replace symbols with words: | Symbol | Word | Example | |-----|-----|-----| | `->` | "then"
| "A then B then C" | | `||` | "and" or "parallel" | "A and B and C" | | `+` | "with" | "A with skill B" |
| `:` | "task" or "on" | "agent task: description" | | `=` | "is" or "equals" | "timeout is 5000" | | `()`
| "group" | "group: A, B" | | `?` | "if" | "if condition" | | `*` | "repeat" | "repeat 3 times" |

3.2 Natural Sentence Structure

Pattern: [Subject] [Action] [Modifiers]

```
Subject:    agent name  
Action:     verb (run, execute, analyze)  
Modifiers:  with, then, and, on, for
```

Examples:

```
"Researcher analyze topic with deep search"
→ researcher : "analyze topic" + deep_search

"Security scan and performance test run together"
→ security_scan || performance_test

"Build then test then deploy"
→ build -> test -> deploy
```

3.3 Command Templates

Template 1: Simple Task

```
[agent] [task-description]

Example:
"API architect design payment API"
→ api-architect : "design payment API"
```

Template 2: Sequential Pipeline

```
[agent1] then [agent2] then [agent3]

Example:
"Research then design then implement"
→ research -> design -> implement
```

Template 3: Parallel Execution

```
parallel: [agent1], [agent2], [agent3]

Example:
"parallel: frontend, backend, database"
→ frontend || backend || database
```

Template 4: Agent with Skills


```
[agent] with [skill1] and [skill2]
```

Example:

```
"API architect with REST patterns and PostgreSQL"  
→ api-architect + rest-patterns + postgresql
```

Template 5: Conditional

```
[action] if [condition] else [fallback]
```

Example:

```
"deploy if tests pass else rollback"  
→ deploy ? test_pass : rollback
```

4. Speech Recognition Patterns

4.1 Phonetic Agent Names

Mapping for speech recognition:

```
phonetic_mappings:  
  "context seven": /ctx7  
  "context lookup": /ctx7  
  "deep researcher": /deep  
  "deep research": /deep  
  "workflow tools": /wflw  
  "orchestrator": /orch  
  "coordinator": /coord  
  "A P I architect": api-architect  
  "A P I designer": api-architect  
  "skill builder": /meta-skill-builder  
  "agent builder": /meta-agent
```

4.2 Wake Words & Trigger Phrases

Activation phrases:

```
"Claude, run workflow..."
"Claude, execute..."
"Claude, start task..."
"Hey Claude..."
```

Examples:

```
"Claude, run workflow: research then design"
→ Activates DSL parser

"Claude, parallel: security scan, performance test"
→ Executes parallel workflow

"Claude, API architect design REST API"
→ Single agent task
```

4.3 Punctuation & Structure (Voice)

Verbal punctuation:

```
"comma"      → ,  (separator)
"period"     → .  (end)
"colon"      → :  (task specification)
"then"       → -> (sequential)
"and"        → || (parallel)
"new line"   → \n (line break)
```

Example:

SPOKEN:

```
"workflow research pipeline colon  
  deep researcher comma then context lookup comma  
  then synthesizer period task colon DSL patterns"
```

PARSED:

```
workflow research_pipeline:  
  deep_researcher, then context_lookup,  
  then synthesizer  
task: DSL patterns
```

FORMAL:

```
workflow research_pipeline {  
  deep_researcher -> context_lookup -> synthesizer  
    : "DSL patterns"  
}
```

5. Natural Language Compiler

5.1 NL-to-DSL Parser Architecture

Speech Input (voice/text)

↓

[Speech-to-Text]

↓

Natural Language Text

↓

[Intent Recognition]

↓

Structured Intent

↓

[DSL Template Matching]

↓

Verbal DSL

↓

[Formal DSL Compiler]

↓

Executable Workflow

5.2 Intent Recognition Patterns

```
class IntentRecognizer:
    patterns = {
        "sequential": [
            r"(.*) then (.*) then (.*)",
            r"(.*) followed by (.*)",
            r"first (.*) then (.*)"
        ],
        "parallel": [
            r"parallel: (.*), (.*), (.*)",
            r"(.*) and (.*) together",
            r"run (.*) and (.*) at the same time"
        ],
        "combination": [
            r"(.*) with (.*) and (.*)",
            r"(.*) plus (.*)",
            r"enhance (.*) with (.*)"
        ],
        "conditional": [
            r"if (.*) then (.*) else (.*)",
            r"(.*) if (.*) otherwise (.*)"
        ],
        "task_spec": [
            r"(.*) task: (.*)",
            r"(.*) on topic (.*)",
            r"(.*) for (.*)$"
        ]
    }

    def recognize(self, text: str) -> Intent:
        for intent_type, patterns in self.patterns.items():
            for pattern in patterns:
                match = re.match(pattern, text, re.IGNORECASE):
                if match:
                    return Intent(
                        type=intent_type,
                        groups=match.groups()
```

```
)  
return Intent(type="unknown")
```

5.3 Template Expansion

```
def expand_to_dsl(intent: Intent) -> str:  
    templates = {  
        "sequential": lambda groups: " -> ".join(groups),  
  
        "parallel": lambda groups: " || ".join(groups),  
  
        "combination": lambda groups: f"{groups[0]} + {' + '.join(groups[1:])}",  
  
        "task_spec": lambda groups: f"{groups[0]} : \"{groups[1]}\""  
    }  
  
    template = templates[intent.type]  
    return template(intent.groups)
```

Example:

```
text = "research then design then implement"  
  
intent = recognize(text)  
# Intent(type="sequential", groups=["research", "design", "implement"])  
  
dsl = expand_to_dsl(intent)  
# "research -> design -> implement"
```

5.4 Ambiguity Resolution

Problem: "and" can mean parallel OR combination **Solution:** Context-based disambiguation

```
def disambiguate_and(text: str) -> str:
    # "A and B" with context clues
    if "together" in text or "parallel" in text:
        return text.replace(" and ", " || ")

    elif "with" in text or "plus" in text:
        return text.replace(" and ", " + ")

    else:
        # Default: assume parallel
        return text.replace(" and ", " || ")
```

Examples:

"Run A and B together"	→ A B (parallel)
"Run A with skill B and C"	→ A + B + C (combination)
"Run A and then B"	→ A -> B (sequential)
"Run A and B"	→ A B (default: parallel)

6. Practical Examples

6.1 Example 1: Simple Research Task

Natural Speech:

```
"Claude, deep researcher analyze quantum computing"
```

Verbal DSL:

```
deep researcher task: analyze quantum computing
```

Formal DSL:

```
deep_researcher : "analyze quantum computing"
```

Execution:

```
| deep_researcher |  
| Task: analyze  |  
| quantum comp.  |
```

6.2 Example 2: Parallel Research

Natural Speech:

```
"Claude, run parallel research streams.  
First stream is deep research,  
second is context lookup for React,  
third is checking documentation.  
Topic is React hooks."
```

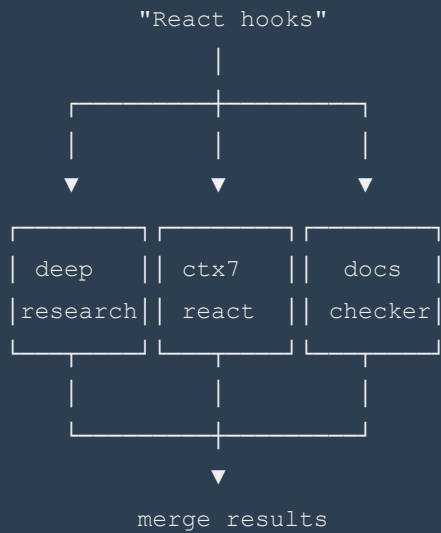
Verbal DSL:

```
parallel:  
  deep research,  
  context lookup for React,  
  check documentation  
topic: React hooks
```

Formal DSL:

```
(  
  deep_researcher ||  
  /ctx7("react") ||  
  docs_checker  
) : "React hooks"
```

Execution:



6.3 Example 3: Sequential Pipeline

Natural Speech:

```
"Claude, run a research pipeline.  
First, research the topic.  
Then, design an API.  
Then, implement it.  
Finally, write tests.  
Topic is payment gateway."
```

Verbal DSL:

```
pipeline:  
  research topic  
  then design API  
  then implement  
  then write tests  
topic: payment gateway
```

Formal DSL:

```
research_agent ->
api_architect ->
implementer ->
test_engineer
  : "payment gateway"
```

Execution:

```
research → design → implement → test
  15 min   20 min   30 min   15 min
Total: 80 minutes sequential
```

6.4 Example 4: Conditional Workflow

Natural Speech:

```
"Claude, run tests.
If all tests pass, deploy to production.
If tests fail, fix the issues and retry.
Maximum three retries."
```

Verbal DSL:

```
run tests
if pass:
  deploy production
else:
  fix issues, then retry up to 3 times
```

Formal DSL:

```
test_suite ->
if(all_pass)
  deploy_production
: (fix_issues -> retry(3))
```

Execution:

```

test_suite
  |
  ?
  /\
pass fail
  |   |
  |   |
deploy fix ↻³

```

6.5 Example 5: Complex Multi-Stream

Natural Speech:

```

"Claude, this is complex.
Run three parallel work streams.

Stream one: combine deep research with context lookup
and general research tools.

Stream two: run orchestration, workflow, and coordination
tools in sequence.

Stream three: run skill builder and agent builder together.

After all streams complete, synthesize everything.
Topic is DSL for agent orchestration."

```

Verbal DSL:

```

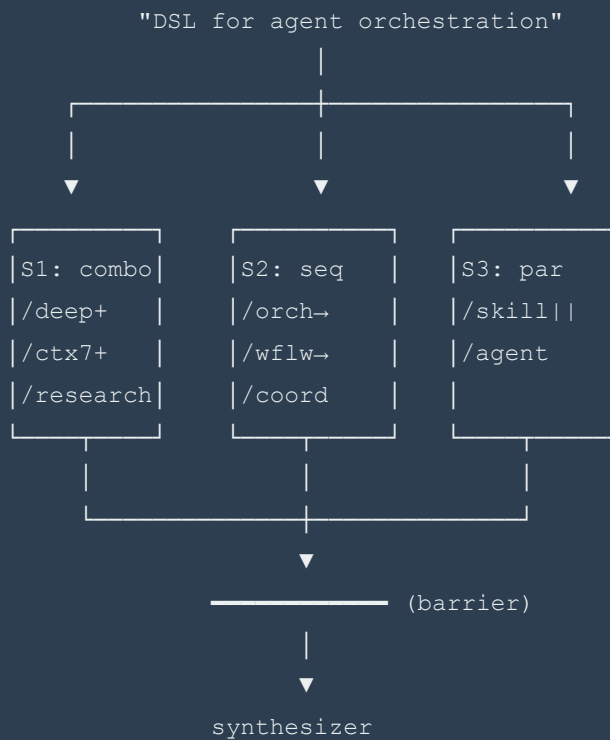
parallel streams:
  stream 1: deep research with context lookup with research tools
  stream 2: orchestration then workflow then coordination
  stream 3: skill builder and agent builder
then: synthesize all findings
topic: DSL for agent orchestration

```

Formal DSL:

```
(
  (/deep + /ctx7 + /research) ||
  (/orch -> /wflw -> /coord) ||
  (/meta-skill-builder || /meta-agent)
) ->
synthesizer
  : "DSL for agent orchestration"
```

Execution:



7. Voice Mode Integration

7.1 Voice-Mode States

Integration with `voice-mode-orchestrator` agent:

```
voice_modes:
  normal:
    activation: "always on"
    audio_feedback: full
    confirmation: required

  afk:
    activation: "manual"
    audio_feedback: minimal
    confirmation: none

  family_time:
    activation: "child voice detected"
    audio_feedback: silent
    notifications: critical only

  focus:
    activation: "manual"
    audio_feedback: silent
    confirmations: none
```

7.2 Voice Commands for Mode Switching

```
"Claude, enter family time mode"
→ Silences all non-critical notifications

"Claude, back to normal mode"
→ Restores full audio feedback

"Claude, focus mode for 2 hours"
→ Silent operation, critical alerts only
```

7.3 Parent-Friendly Workflow

Scenario: Working parent needs to run complex workflow while caring for child **Natural Speech** (quiet voice):

```
"Claude, run research workflow, family time mode.  
Parallel: deep research, context lookup, documentation.  
Only notify me if critical errors.  
Topic: React hooks tutorial."
```

Verbal DSL:

```
mode: family_time  
notify: critical_only  
parallel: deep research, context lookup, documentation  
topic: React hooks tutorial
```

Formal DSL:

```
workflow_config:  
  mode: family_time  
  notifications: critical_only  
  
workflow:  
  (/deep || /ctx7 || /docs) : "React hooks tutorial"
```

Execution: - Runs silently in background - No audio confirmations - Only surfaces critical failures - Preserves context if interrupted ---

8. Accessibility Features

8.1 Screen Reader Support

Verbose mode for screen readers:

```
Standard:  "research -> design -> implement"
Verbose:   "Sequential workflow: research agent,
           followed by design agent,
           followed by implement agent"

Standard:  "a || b || c"
Verbose:   "Parallel execution of three agents:
           agent a, agent b, and agent c"
```

8.2 Motor Impairment Support

Simplified command structure:

```
Instead of:  "/deep + /ctx7 || /orch -> synthesize : task"

Use:         "run:
              parallel: deep, context
              sequence: orchestrator, synthesize
              task: description"
```

8.3 Cognitive Load Reduction

Progressive disclosure:

```
Level 1 (Beginner):
  "Run researcher on quantum computing"

Level 2 (Intermediate):
  "Run researcher then designer on quantum computing"

Level 3 (Advanced):
  "Parallel: researcher, context lookup
  Then: designer
  Topic: quantum computing"

Level 4 (Expert):
  Formal DSL
```

9. Quick Reference Cards

Card 1: Basic Voice Commands

BASIC VOICE COMMANDS

Single Task:

"Agent-name do task"

Sequential:

"A then B then C"

Parallel:

"Parallel: A, B, C"

With Skills:

"Agent with skill1 and skill2"

Conditional:

"If condition then A else B"

Card 2: Agent Pronunciations

HOW TO SAY AGENT NAMES

/ctx7	→ "context seven"
/deep	→ "deep research"
/orch	→ "orchestrator"
/wflw	→ "workflow tools"
/coord	→ "coordinator"
api-architect	→ "A P I architect"
/meta-skill	→ "skill builder"
/meta-agent	→ "agent builder"

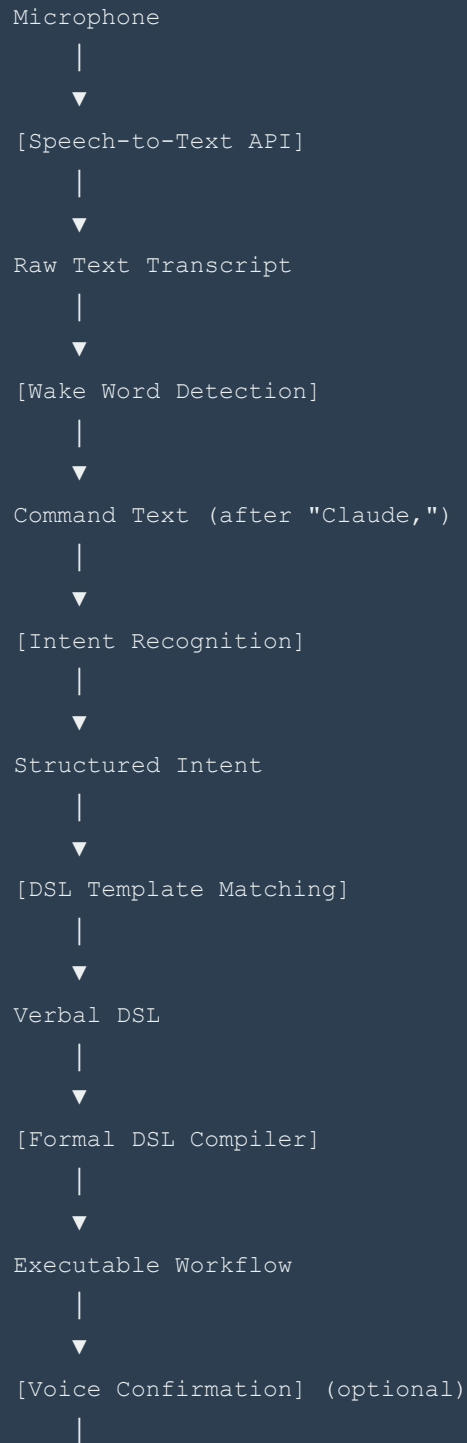
Card 3: Keyword Translations

SPEAK → DSL KEYWORDS

"then"	→ ->	(sequential)
"and"	→	(parallel)
"with"	→ +	(combination)
"task"	→ :	(specification)
"equals"	→ =	(assignment)
"if...else"	→ ? :	(conditional)
"retry"	→ ∞	(retry)
"repeat"	→ *	(iteration)

10. Implementation Guide

10.1 Voice Pipeline Architecture





Execute

10.2 Sample Implementation (Python)

```
from anthropic import Anthropic
import speech_recognition as sr

class VoiceDSLCompiler:
    def __init__(self):
        self.recognizer = sr.Recognizer()
        self.client = Anthropic()

    def listen(self) -> str:
        """Capture speech and convert to text"""
        with sr.Microphone() as source:
            audio = self.recognizer.listen(source)
            return self.recognizer.recognize_google(audio)

    def parse_intent(self, text: str) -> dict:
        """Use Claude to parse natural language intent"""
        response = self.client.messages.create(
            model="claude-3-sonnet-20240229",
            messages=[{
                "role": "user",
                "content": f"""Parse this voice command into structured intent:
                "{text}"

                Return JSON with:
                - type: sequential | parallel | combination | task_spec
                - agents: list of agent names
                - task: task description (if any)
                """
            }]
        )
        return parse_json(response.content)

    def compile_to_dsl(self, intent: dict) -> str:
        """Convert structured intent to formal DSL"""
        if intent["type"] == "sequential":
            return " -> ".join(intent["agents"])

        elif intent["type"] == "parallel":
            return " || ".join(intent["agents"])
```

```

elif intent["type"] == "task_spec":
    return f"{intent['agents'][0]} : \"{intent['task']}\""}

# ... other cases

def execute(self):
    """Main voice loop"""
    print("Listening for voice commands...")

    while True:
        try:
            text = self.listen()

            if not text.lower().startswith("claude"):
                continue # Ignore non-commands

            # Remove wake word
            command = text[7:].strip()

            # Parse and compile
            intent = self.parse_intent(command)
            dsl = self.compile_to_dsl(intent)

            # Confirm
            print(f"DSL: {dsl}")
            print("Execute? (yes/no)")

            confirmation = input()
            if confirmation.lower() == "yes":
                # Execute workflow
                result = self.execute_workflow(dsl)
                print(f"Result: {result}")

        except Exception as e:
            print(f"Error: {e}")

# Usage
compiler = VoiceDSLCompiler()
compiler.execute()

```

Summary

This verbal interface makes advanced DSL orchestration **accessible through natural language**: 1. **Natural speech** → **Verbal DSL** → **Formal DSL** translation 2. **Voice-friendly syntax** with speakable keywords 3. **Intent recognition** for command parsing 4. **Accessibility features** for diverse needs 5. **Family time integration** with voice-mode-orchestrator 6. **Progressive disclosure** from simple to complex **Key Innovation**: Complex mathematical workflows become as simple as:

```
"Claude, run deep research and context lookup in parallel,  
then synthesize findings on quantum computing."
```

--- **Next Steps**: 1. Implement speech-to-text integration 2. Train intent recognition model 3. Build DSL compiler 4. Integrate with voice-mode-orchestrator 5. Test with working parents use case --- **Version**: 1.0.0 **Status**: Specification Complete **Target**: Voice-first agent orchestration
✓